

Python para Ciência de Dados

Print to PDF

Contents

- Manipulação de Dados Tabulares com Pandas - Funções Básicas
- Tipos de Dados com Pandas
- Renomeando Colunas
- Seleccionando Linhas e Colunas
- Adicionando e Removendo Colunas
- Operações Básicas - Consultas
- Operações Básicas - Ordenação
- Operações Básicas - Combinando DataFrames
- Operações Básicas - Salvando DataFrames

Manipulação de Dados Tabulares com Pandas - Funções Básicas

```
import pandas as pd
```

```
idades = pd.DataFrame(  
    [  
        ['Paraná', 'Londrina', 575377, 1356.00],  
        ['São Paulo', 'São Carlos', 254484, 1508.00],  
        ['Santa Catarina', 'Florianópolis', 508826, 1798.00],  
        ['Paraná', 'Curitiba', 1963726, 2293.00],  
        ['São Paulo', 'Campinas', 1223237, 1710.00]  
    ], columns=['Estado', 'Cidade', 'Habitantes', 'Salário-Médio'])  
  
idades
```

```
df = pd.read_csv('qualidade_vinho.csv', sep=',', header=0)  
df.head(2)
```

```
idades.head(2) # Retorna um novo dataframe com as n primeiras amostras
```

```
idades.tail(2) # Retorna um novo dataframe com as n últimas amostras
```

```
idades.shape # Retorna uma tupla com o número de linhas e colunas
```

```
idades.info() # Retorna vazio e imprime na tela as informações gerais do dataframe
```

```
idades.columns # Retorna uma lista com o nome de todas as colunas
```

```
idades['Salário-Médio'].describe() # Retorna um objeto DataFrame com as medidas es
```

Tipos de Dados com Pandas

```
idades.dtypes  
idades.info()
```

```
idades['Habitantes'] = idades['Habitantes'].astype('float64')  
idades.dtypes
```

```
idades['Data'] = ['3-8-2021', '3-9-2021', '3-10-2021', '3-11-2021', '3-12-2021']  
idades['Data'] = pd.to_datetime(idades['Data'], format='%d-%m-%Y') # Dia, Mês, Ano  
idades
```

```
idades['Data'] = idades['Data'].dt.strftime('%d-%m-%Y') # Dia, Mês, Ano  
idades # Novo Formato
```

Renomeando Colunas

```
idades.rename(columns={'Habitantes': 'N Habitantes', 'Data': 'Data - Censo'}, inplace=True)
idades
```

Selecionando Linhas e Colunas

```
idadesHab = idades[['N Habitantes', 'Salário-Médio']] # Seleciona as duas colunas
idadesHab
```

```
idades.loc[[1, 2]] # Seleciona as duas linhas com os nomes especificados (note que
```

```
idades.iloc[1:3] # Seleciona as linhas do dataframe original, referentes ao índice
```

```
idades.iloc[:, 0:2] # Seleciona todas as linhas e as duas primeiras colunas do data
```

```
idades.iloc[1:3, :3] # Seleciona as duas primeiras linhas e as três primeiras colu
```

```
idades.iloc[:, [1, 3, 0, 2]] # Seleciona todas as linhas e as colunas determinadas
```

Adicionando e Removendo Colunas

```
idades.insert(2, 'Sigla', ['PR', 'SP', 'SC', 'PR', 'SP']) # Adiciona novos dados no
idades.insert(2, 'Sigla-Repetida', idades['Sigla']) # Adiciona uma coluna repetida
idades
```

```
idades.pop('Sigla-Repetida')

# ou

idades = idades.drop(['Sigla-Repetida'], axis=1)
idades
```

Operações Básicas - Consultas

```
idades.query('`Salário-Médio` > 1400.0')
```

```
idades.query('Estado == "São Paulo"')
```

```
idades.query('`N Habitantes` > 500000 and `N Habitantes` < 1300000')
```

Operações Básicas - Ordenação

```
idades.sort_values(by='Cidade', inplace=True)  
idades
```

```
idades.sort_values(by='Salário-Médio', inplace=True)  
idades
```

Operações Básicas - Combinando DataFrames

```
idades_salarios = pd.DataFrame(  
    [  
        ['Paraná', 'Londrina', 1356.00],  
        ['São Paulo', 'São Carlos', 1508.00],  
        ['Santa Catarina', 'Florianópolis', 1798.00],  
        ['Paraná', 'Curitiba', 2293.00],  
        ['São Paulo', 'Campinas', 1710.00]  
    ], columns=['Estado', 'Cidade', 'Salário-Médio'])  
  
idades_salarios
```

```
idades_pop = pd.DataFrame(  
    [  
        ['Londrina', 575377],  
        ['São Carlos', 254484],  
        ['Florianópolis', 508826],  
        ['Curitiba', 1963726],  
        ['Campinas', 1223237]  
    ], columns=['Cidade', 'Habitantes'])  
  
idades_pop
```

```
pd.merge(left=cidades_salarios, right=cidades_pop, on="Cidade")
```

```
pd.merge(left=cidades_salarios, right=cidades_pop, on="Cidade", how="outer")
```

```
idades_df1 = pd.DataFrame(  
    [  
        ['Londrina', 575377],  
        ['São Carlos', 254484],  
        ['Florianópolis', 508826],  
    ], columns=['Cidade', 'Habitantes'])  
  
idades_df1
```

```
idades_df2 = pd.DataFrame(  
    [  
        ['Curitiba', 1963726],  
        ['Campinas', 1223237]  
    ], columns=['Cidade', 'Habitantes'])  
  
idades_df2
```

```
pd.concat([idades_df1, idades_df2], ignore_index=True)
```

Operações Básicas - Salvando DataFrames

```
idades.to_csv("my_df.csv")  
idades.to_html("my_df.html")  
idades.to_json("my_df.json")
```